

Integración de Servicios Cloud GCP

Interfaces de red, servicios, alias y herramientas de IA en Linux

Semana 03 - Sesión 05

Rel Guzman



**Universidad
Tecnológica
del Perú**

- ▶ ¿Qué diferencia existe entre la aplicación Terminal y la shell?
- ▶ ¿Qué información muestran `whoami`, `hostname` y `pwd`?
- ▶ ¿Por qué se ejecuta `sudo apt update` antes de instalar un paquete?
- ▶ ¿Cómo crearías una carpeta, entrarías en ella y comprobarías tu ubicación?

Logro de la sesión

Al finalizar, podrás interpretar la red de la MV, publicar y controlar servicios, crear atajos persistentes en Bash e instalar herramientas de IA para solicitar ayuda desde la terminal.

- ▶ ¿Qué identifica una dirección IP dentro de una red?
- ▶ ¿Qué diferencia imaginas entre ejecutar un programa y mantenerlo como servicio?
- ▶ ¿Qué ocurriría si un servidor solo escucha en 127.0.0.1?
- ▶ ¿Cómo evitarías escribir varias veces el mismo comando?

¿Para qué te servirá lo aprendido?

Podrás comprobar cómo se comunica Ubuntu, publicar el servidor Node de la práctica, controlar el inicio de servicios y reutilizar comandos frecuentes.

También podrás instalar Warp y Antigravity CLI para pedir explicaciones y apoyo sobre el proyecto, revisando siempre los comandos y cambios sugeridos.

Interfaces de red

Definición inicial

Una **interfaz de red** es la parte que Ubuntu usa para comunicarse con otros equipos.

Cómo reconocerla

Ubuntu le asigna un nombre. En nuestra MV veremos `enp0s1`.

Qué ocurre por ella

Por la interfaz salen solicitudes y regresan respuestas.

Ejemplo sencillo

Al abrir un sitio web, la solicitud sale de Ubuntu por `enp0s1` y la respuesta regresa por esa misma interfaz.

Resultado simplificado

```
rgap@ubuntu:~$ ip -br addr
lo UNKNOWN 127.0.0.1/8
enp0s1 UP 192.168.100.158/24
```

Dos direcciones, dos usos

127.0.0.1 funciona solo dentro de Ubuntu.

192.168.100.158 identifica a enp0s1 en la red virtual.

Adaptado a esta máquina virtual

La MV no usa una interfaz Wi-Fi propia. Solo vemos lo y enp0s1; la Mac gestiona el Wi-Fi fuera de Ubuntu.

Un servidor HTTP mínimo combina dirección y puerto

Archivo completo: `server.js`

```
const http = require('http');
const server = http.createServer((req, res) => {
  res.end('Hola desde Ubuntu');
});
server.listen(8000, '0.0.0.0', () => {
  console.log('Servidor listo');
});
```

No requiere paquetes adicionales

El módulo `http` ya viene incluido con Node.js.

Qué hace

`http` crea el servidor.

`res.end` envía la respuesta.

8000 es el puerto.

0.0.0.0 permite escuchar por lo y enp0s1.

127.0.0.1 Solo acepta conexiones originadas dentro de la propia MV.

192.168.100.158 Acepta conexiones dirigidas específicamente a enp0s1.

0.0.0.0 Escucha en todas las interfaces disponibles.

Ejecuta el servidor

```
$ node server.js  
Servidor listo
```

Prueba en el navegador

Dentro de la MV:

`http://127.0.0.1:8000`

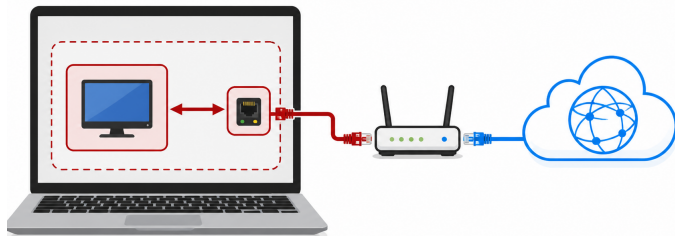
Por enp0s1:

`http://192.168.100.158:8000`

Si funciona con localhost pero no con la IP

Revisa primero la dirección de escucha y el firewall. Después comprueba el adaptador virtual y la red.

El adaptador virtual enlaza la MV con la red externa



1. La MV usa su interfaz.
2. El adaptador virtual entrega el tráfico al hipervisor.
3. El router permite salir de la red local.
4. La conexión alcanza internet.

Líneas principales del resultado

```
rgap@ubuntu:~$ ip a
1: lo
   inet 127.0.0.1/8 scope host lo
2: enp0s1
   inet 192.168.100.158/24
   scope global dynamic enp0s1
```

Lectura rápida

UP: enp0s1 está activa.

dynamic: Ubuntu recibió la dirección IP automáticamente.

UNKNOWN en lo: no es un error; lo funciona solo dentro de Ubuntu.

`ip a` Lista interfaces y direcciones.

`lo` Interfaz interna del propio Ubuntu.

`enp0s1` Conecta la MV con la red.

`inet` Presenta una dirección IPv4: cuatro números separados por puntos.

`/24` Indica la parte que identifica la red local.

Una es interna y la otra conecta con la red

El resultado de `ip a` presenta dos interfaces con funciones diferentes.

`lo`: comunicación interna

Dirección: `127.0.0.1/8`.

Ubuntu la usa para comunicarse consigo mismo. No conecta la MV con otros equipos.

`enp0s1`: comunicación con la red

Dirección: `192.168.100.158/24`.

Es la interfaz que permite que la MV se comunique con la red externa.

Lectura correcta del resultado

Aunque `ip a` muestra dos interfaces, solo `enp0s1` conecta la MV con la red externa. `lo` funciona únicamente dentro de Ubuntu.

Ruta Regla que indica por dónde debe enviarse el tráfico hacia un destino.

Puerta de enlace Router que permite salir de la red local; también se denomina *gateway*.

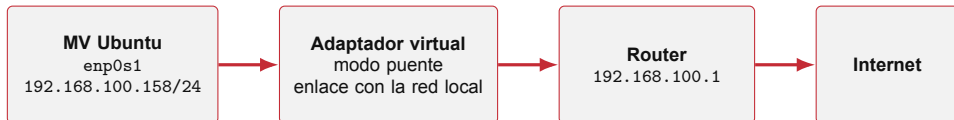
DNS Servicio que traduce nombres, como `example.com`, a direcciones IP.

Ejemplo completo

En este laboratorio, la MV usa `192.168.100.158/24` y la Mac `192.168.100.140/24`. Ambas están en la misma red local. La puerta de enlace `192.168.100.1` permite salir hacia internet y el DNS resuelve el nombre del sitio.

El modo puente ubica la MV en la misma red que la Mac

La MV conserva su propia dirección, 192.168.100.158/24. El adaptador virtual enlaza `enp0s1` con la red local y el router recibe el tráfico directamente.



Flujo completo

El DNS convierte el nombre del sitio en una IP. La solicitud sale por `enp0s1`, cruza el adaptador virtual y llega al router. La respuesta regresa por el mismo camino hasta la MV.

La Mac y la MV comparten la red 192.168.100.0/24, pero cada una conserva su propia dirección.

Servicios en Linux

Qué es un servicio

Es un programa que funciona en segundo plano para atender una tarea, como publicar una página web.

`systemd`

Es el componente de Ubuntu que inicia, detiene y supervisa los servicios del sistema.

`systemctl`

Es el comando que usamos para consultar y controlar esos servicios desde la terminal.

Ejemplo para esta MV

En Ubuntu, el servidor web Apache se administra como `apache2.service`. El nombre `httpd.service` se utiliza en otras distribuciones.

Consulta inicial

```
$ systemctl status apache2
apache2.service - The Apache HTTP Server
Loaded: loaded (...; disabled)
Active: inactive (dead)
```

Cómo leerlo

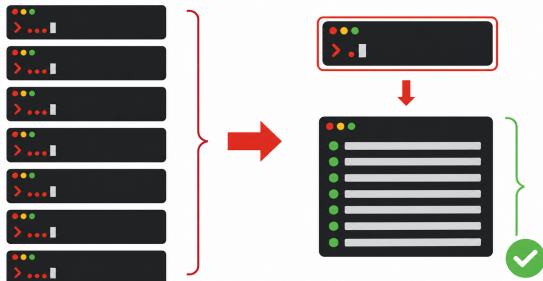
inactive: Apache está detenido. disabled: no

arrancará automáticamente al encender la MV.

Un comando puede estar bien escrito y aun así fallar si el servicio indicado no existe.

- status** Muestra el estado.
- start** Inicia ahora.
- enable** Activa el inicio automático.
- stop** Detiene ahora.
- disable** Desactiva el inicio automático.

Alias y funciones



Alias

Nombre corto que representa **un comando** con sus opciones. Ejemplo: o puede representar `xdg-open ..`

Función

Nombre que agrupa **uno o varios comandos** ejecutados en orden.
Forma: `nombre() { comandos; }`.

Duración en esta práctica

Ambos existen solo en la terminal actual. Al cerrarla, desaparecen.

Bash y Zsh son shells; \$SHELL indica la configurada

Variable de entorno

Dato con nombre que la shell conserva para describir la sesión. SHELL guarda la ubicación completa de la shell configurada. El comando `echo` muestra un texto o el valor de una variable.

Bash en Ubuntu

```
rgap@ubuntu:~$ echo $SHELL  
/bin/bash
```

Zsh en otra cuenta

```
rel@equipo:~$ echo $SHELL  
/bin/zsh
```

Resultado: ambas son shells; la salida permite reconocer cuál está configurada.

Dos elementos antes de crear el alias

xdg-open solicita a Ubuntu abrir un elemento con la aplicación gráfica predeterminada. El punto . representa el directorio actual.

Comando completo

```
rgap@ubuntu:~$ pwd  
/home/rgap  
rgap@ubuntu:~$ xdg-open .
```

Resultado visible

Ubuntu abre la aplicación **Archivos** y muestra el contenido de /home/rgap, que era el directorio actual.

Idea del alias: escribir o será más rápido que repetir xdg-open ..

Parte A: Apache y Node

Si falta Apache: `sudo apt install apache2 -y`.

1. Consulta: `systemctl status apache2`.
2. Inicia: `sudo systemctl start apache2`.
3. Habilita: `sudo systemctl enable apache2`;
abre `http://192.168.100.158`.
4. Detén: `sudo systemctl stop apache2`.
5. Deshabilita: `sudo systemctl disable apache2`.
6. Ejecuta `node server.js` y prueba el puerto 8000.

Comprobación final: `is-active` e `is-enabled` muestran `inactive` y `disabled`; Node responde en `192.168.100.158:8000`; o abre la carpeta e `info` muestra usuario, equipo y ruta.

Parte B: alias y función

1. Confirma la shell con `echo $SHELL`.
2. Crea alias `o='xdg-open .'` y ejecuta `o`.
3. Crea `info() { whoami; hostname; pwd; }`.
4. Ejecuta `info` y verifica sus tres datos.

Archivo de configuración de Bash

.bashrc es un archivo oculto ubicado en el directorio personal. Bash lee sus instrucciones cada vez que inicia una terminal interactiva.

Solo en la terminal actual

Si creas o `echo` directamente en la terminal, desaparecen cuando la cierras.

Guardados en .bashrc

Si escribes allí el alias y la función, Bash vuelve a crearlos al abrir cada terminal nueva.

Dos comandos que utilizaremos

`nano ~/.bashrc` abre el archivo en un editor de texto. `source ~/.bashrc` vuelve a leerlo sin cerrar la terminal actual.

1. Abre el archivo con `nano ~/.bashrc`.
2. Agrega el alias `o` y la función `info` al final.
3. Guarda con `Ctrl+O`, confirma con `Enter` y sal con `Ctrl+X`.
4. Ejecuta `source ~/.bashrc`.
5. Prueba `o` e `info`.

Contenido y resultado esperado

```
$ nano ~/.bashrc
alias o='xdg-open .'
info() { whoami; hostname; pwd; }
$ source ~/.bashrc
$ o
Se abre la carpeta actual en Archivos
$ info
rgap
ubuntu
/home/rgap/curso_cloud/sesion05
```

Comprobación: abre una terminal nueva y ejecuta `o` e `info`; ambos deben seguir disponibles.

Ayuda con IA en la terminal

Para qué sirve

Warp es una terminal gráfica que puede explicar comandos, sugerir pasos y ayudar a trabajar con proyectos. En Ubuntu puede seguir utilizando Bash.

1. Comprueba la arquitectura

```
$ dpkg --print-architecture  
arm64
```

La salida también puede ser amd64.
Descarga el paquete .deb
correspondiente.

Úsalo como apoyo

Puedes pedir una explicación de `systemctl status apache2` antes de ejecutarlo. Revisa siempre el comando y confirma que corresponda a tu MV.

2. Instala y abre Warp

```
$ cd ~/Descargas  
$ sudo apt install ./warp-terminal_*.deb  
$ warp-terminal
```

Descarga primero el paquete para Debian/Ubuntu
desde warp.dev/download.

Para qué sirve

agy abre un agente de IA en la terminal. Puede revisar archivos del proyecto, explicar código y proponer cambios sujetos a tu confirmación.

Instalación oficial en Ubuntu

```
$ sudo apt update
$ sudo apt install curl -y
$ curl -fsSL \
  https://antigravity.google/cli/install.sh | bash
$ source ~/.bashrc
```

Ejemplo en el laboratorio

```
$ cd ~/curso_cloud/sesion05
$ agy
```

Solicitud de ejemplo:

Explica server.js y dime cómo probarlo.

Primer inicio y ruta del comando

El primer inicio puede abrir el navegador para iniciar sesión. Si aparece agy: `command not found`, agrega `export PATH="$HOME/.local/bin:$PATH"` a `~/.bashrc` y ejecuta `source ~/.bashrc`.



gracias



Universidad
Tecnológica
del Perú

- ▶ Debian Administrator's Handbook y páginas de manual: `bash`, `ip`, `systemctl`, `systemd.service`, `xdg-open` y `nano`.
- ▶ Documentación de Node.js: módulo nativo `http` y método `server.listen`.
- ▶ Documentación del hipervisor utilizado: adaptadores virtuales y modo de red puente.
- ▶ Documentación oficial de Warp: instalación y configuración en Linux, `docs.warp.dev`.
- ▶ Documentación oficial de Google Antigravity CLI: descripción, instalación y autenticación, `antigravity.google/docs/cli`.
- ▶ Material base proporcionado: *Fundamentos de Linux — Programa completo*.
- ▶ Capturas proporcionadas en `Sesion05/diapos-imagenes`, simplificadas para la MV del laboratorio; ilustraciones de red virtual y alias/función.